

Advanced Information Security

V0087/V0088

Instructor: Xiaohong Yu

Date: May 14, 2026

Due: May 28, 2026

Total Credits: 100 pts

<Problem Solving>

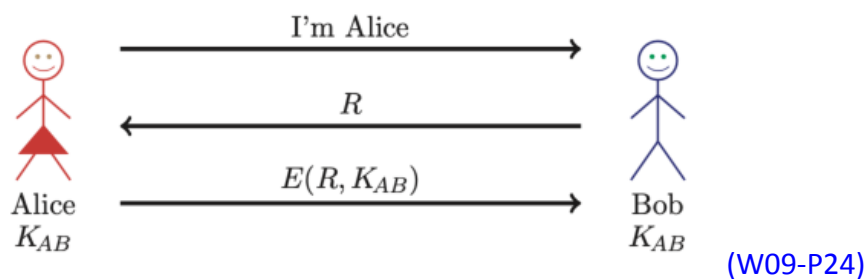
1. 简单安全协议 [25points]

a. 已知一个“对称密钥双向认证”方案是把单向 challenge-response 协议直接执行两次来实现互认证。[10points]

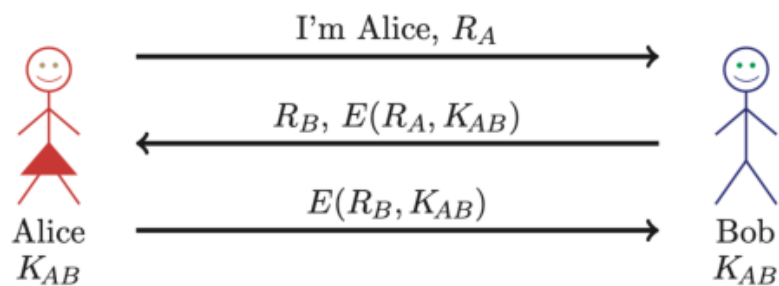
- 1). 画出消息流程 (Alice、Bob、Trudy)。[3 points]
- 2). 说明 Trudy 如何利用反射攻击通过认证。指出该方案失败的根本原因。[3points]
- 3). 提出一种最小修改方案并解释为什么能防住反射攻击。[4 points]

Answer:

使用对称密钥进行单向认证的质询-应答协议方式如下图：

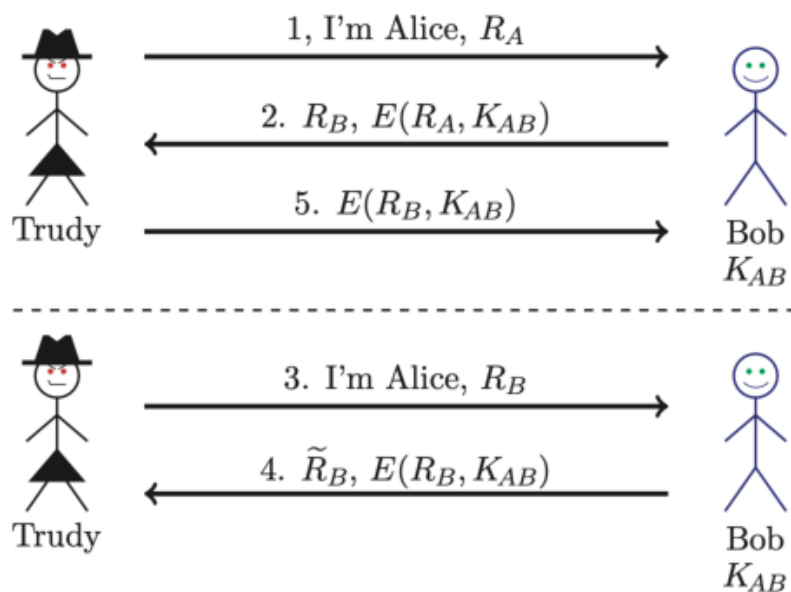


1) 使用对称密钥进行双向认证, 并按照单向认证执行两次来实现相互认证, 对应的消息流程为：



(W09-P27)

使用单向认证重复两次完成对应的双向认证容易受反射攻击，以下为Trudy假装Alice与Bob完成双向认证的流程：



(W09-P28)

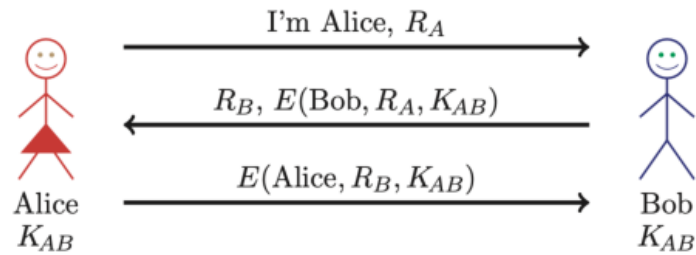
2) 如上图所示，Trudy 假冒Alice与Bob 建立连接，并首先发出(1)质询随机数 R_A ，然后Bob回复(2)经过共同密钥加密的 $E(R_A, K_{AB})$ 应答，同时发出验证Alice的质询随机数 R_B ，但是Trudy不知道Alice和Bob的共享密钥 K_{AB} ，所以Trudy没办法回复。Trudy选择与Bob重新创建一个新连接(3)，并发送对应的 R_B 给Bob进行质询，于是Bob针对新的连接重新回复了对应的(4)基于 R_B 的 $E(R_B, K_{AB})$ 。Trudy收到 $E(R_B, K_{AB})$ 应答后，可以继续完成第一次建立连接时的Bob的质询，(5) 回复 $E(R_B, K_{AB})$ 给Bob,从而完成第一次连接的双向认证。

该方案的主要问题是双向认证直接重复单向协议，双方使用相同共享密钥，两个方向的认证过程完全对称，且缺乏角色或方向区分。攻击者可将一方的 challenge 反射到另一会话中，诱使合法参与方计算响应并复用，从而通过认证。本质上是协议缺少防止反射攻击的机制。

3) (给出一种方案就可分)

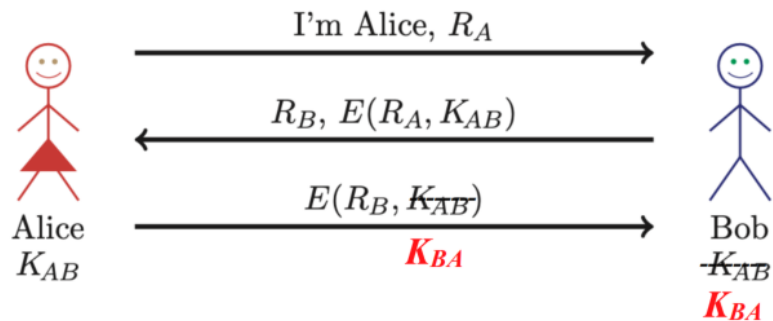
最小修改方案1: 通过发送应答时, 添加发送人身份信息可以有效防止上述提到的反射攻击。

具体方案为:

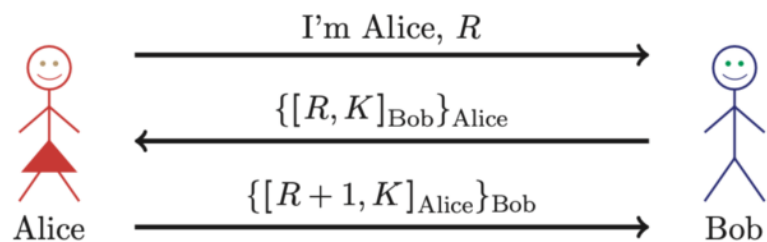


(W09-P30)

最小修改方案2: 双方使用不同的密钥 K_{AB} 和 K_{BA} , 进行双向认证。



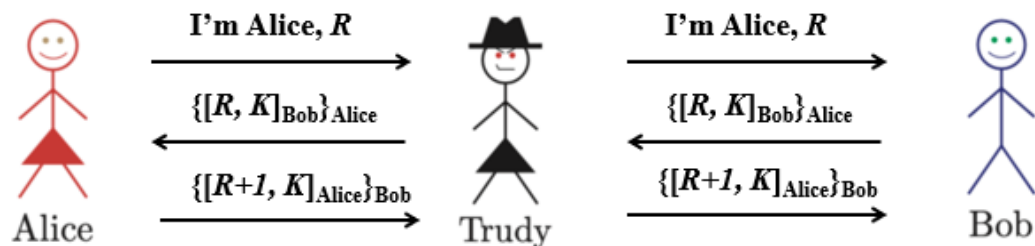
b. 假设 Alice 和 Bob 打算通过一个不安全的网络建立一个共享的会话密钥 K , 以便后续进行加密通信。他们设计了如下图所示的认证与密钥协商协议。[15points]



- 1). 外层的公钥加密(如 $\{\dots\}_{\text{Alice}}$)主要提供了哪种安全属性? 保护了图中的哪个关键元素? [3points]
- 2). 内层的数字签名(如 $[\dots]_{\text{Bob}}$)主要提供了哪些安全属性? [3 points]
- 3). 仔细观察该协议, 它虽然使用了加密和签名, 但存在一个严重的逻辑漏洞。请写出当前协议可能遭受哪种攻击, 并描述攻击步骤。 [4 points]
- 4). 请给出一种方案修复上述漏洞。 [5 points]

Answer:

- 1) 外层的公钥加密主要提供了消息的机密性。只有接收方使用自己的密钥能够解密消息内容。保护了图中的会话密钥K和随机数R。
- 2) 内层的数字签名提供了数据的完整性, 同时提供了身份验证, 通过内部签名, 接收方可以使用发送方的公钥进行签名验证, 同时验证数据有没有被篡改, 和是否的发送方的签名。通过发送方的签名还可以提供不可否认性。
- 3) 此协议可能会遭受中间人攻击。



攻击者 Trudy 位于 Alice 与 Bob 之间, 分别伪装成对方与双方建立连接, 从而在不掌握会话密钥内容的情况下控制通信过程。

具体攻击步骤如下:

- a. Alice 发起认证请求

Alice 向 Bob 建立连接并发送随机数 challenge: I'm Alice, R

此时, 中间人 Trudy 截获该消息, 并假装自己是 Bob 与 Alice 通信; 同时, Trudy 再伪装成 Alice, 与 Bob 建立新的连接会话。

b. Bob 返回认证响应与会话密钥

Bob 生成会话密钥 K , 并发送: $\{[R, K]_{\text{Bob}}\}_{\text{Alice}}$

由于消息是使用 Alice 公钥加密的, Trudy 无法解密得到 K 。但是, Trudy 无需解密, 只需原样转发给 Alice。

Alice 解密后验证 Bob 的签名成功, 并检查随机数 R 正确, 因此接受会话密钥 K , 默认认为通信对象是真正的 Bob。

c. Alice 完成反向认证

Alice 验证成功后, 发送: $\{[R+1, K]_{\text{Alice}}\}_{\text{Bob}}$

该消息实际先到达 Trudy。Trudy 同样无需解密, 仅原样转发给 Bob。

Bob 解密并验证签名成功, 确认 $R+1$ 正确后, 认为对方确实是 Alice。

最后: Alice 认为自己正在与 Bob 通信, Bob 也认为自己正在与 Alice 通信, 但实际上所有通信都经过 Trudy 转发。

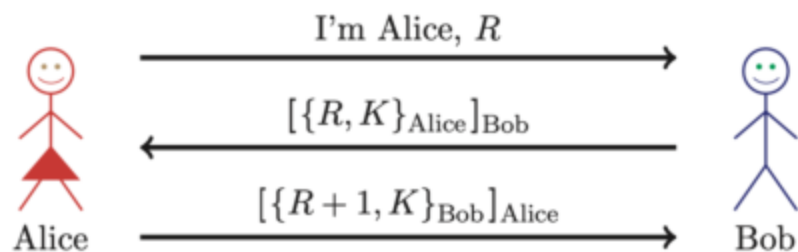
虽然协议同时使用了 数字签名(保证身份真实性与完整性) 和 公钥加密(保证机密性), 但由于协议消息中 缺少对通信双方身份的绑定(identity binding), 攻击者仍可通过消息转发隐藏在双方之间, 实现连接控制、会话重定向以及流量劫持等中间人攻击行为。

4). 现在的协议虽然使用了数字签名和公钥加密, 但仍可能遭受中间人攻击, 因为攻击者可以原样转发消息完成认证。

答案1:

当前消息内容是对随机数 R , 和密钥 K 进行了签名, 没有形成身份绑定, 然后进行了加密。

根据课件内容(W09)可以更改签名和加密顺序(由先签名后加密改为先加密后签名)来初步防止上述中间人攻击。

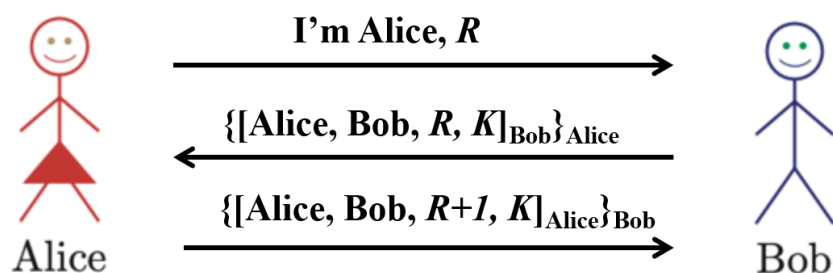


修改后，如上图所示，随机数 R 和会话密钥 K 会先使用 Alice 的公钥加密，形成仅 Alice 能解密的密文，再由 Bob 对该密文进行签名。由于签名对象变成了“发送给 Alice 的密文”，消息实际上与接收者身份形成了隐式绑定，即表明该消息是专门发送给 Alice 的。

因此，中间人攻击者虽然仍可截获并转发消息，但无法将 Bob 发给 Alice 的消息替换为发给自己的消息，也无法修改接收者身份，否则会导致 Bob 的签名验证失败。相比原协议，这种方式降低了认证消息被跨身份复用的风险，从而能够防止之前协议中的简单中间人利用方式。

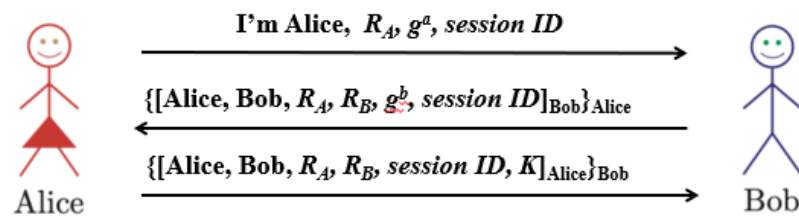
答案2:

当前协议是对随机数 R 和密钥 K 进行了签名，没有形成身份绑定，然后进行了加密。容易遭受中间人攻击，通过原样转发消息完成认证。为了防止中间人攻击，可以通过在协议中添加身份绑定来实现。



答案3:

根据课件W10中现实安全协议中防止中间人攻击的预防方式(P13)，将会话ID，双方身份，临时公钥，双方随机数等全部绑定进签名。



2. 现实安全协议 [25points]

a. 早期的安全协议在设计时, 受限于当时的计算能力或对安全威胁的认知不足, 存在严重的设计缺陷。请分析以下三个早期协议/标准的致命弱点, 并说明现代协议是如何修复这些问题的: WEP (无线局域网保密协议), GSM (2G 移动通信网络) 和SSL (安全套接字层协议) [10points]

Answer:

1) WEP 是早期无线局域网的保密协议, 受限于当时的硬件计算能力, 其采用 RC4 流密码进行加密, 并仅使用 24-bit 的初始化向量 (IV)。由于 IV 长度过短, 在大量通信场景下容易发生重复, 导致:

相同密钥流 (keystream) 被重复使用。

攻击者可利用这一特点, 通过抓包和统计分析 (如 FMS 攻击) 恢复共享密钥。此外, WEP 使用静态共享密钥, 缺乏强身份认证与完整性保护机制, 进一步降低了协议安全性。

基于上述缺陷, 现代无线协议 (如 WPA2/WPA3) 进行了改进:

使用 AES-CCMP 替代 RC4, 提高加密强度;

使用更长随机数与动态会话密钥, 避免密钥流复用问题;

引入更强的身份认证机制 (如 802.1X/EAP) 实现用户级认证;

增强数据完整性保护。

因此, 现代无线协议显著提升了无线网络的安全性。

2) GSM (2G 移动通信网络)

GSM 是早期的 2G 移动通信网络, 其最大的安全缺陷在于: 仅提供单向认证 (One-way Authentication)。

即手机终端需要向网络证明身份, 但网络基站无需向手机证明真实性, 因此攻击者可搭建: 伪基站 (Fake Base Station / IMSI Catcher) 诱骗用户终端连接。此外, GSM 使用的加密算法 A5/1、A5/2 安全性较弱, 容易受到破解攻击。

基于这些缺陷, GSM 常见的安全问题包括:

伪基站攻击；
通话与短信窃听；
用户身份信息(IMS)泄露；
强制关闭加密或降级加密攻击。

为解决上述问题，现代 3G、4G、5G 网络进行了改进：

引入 双向认证(Mutual Authentication)，实现终端与网络的相互身份验证；
使用更强加密算法(如 AES、SNOW 3G、ZUC)；
引入动态会话密钥管理机制。

从而有效降低了伪基站、窃听与身份冒充等安全风险。

3) SSL(Secure Sockets Layer, 安全套接字层协议)

早期 SSL(尤其 SSL 2.0 和 SSL 3.0)存在多项安全缺陷。一方面，其依赖的加密算法(如 RC4、MD5、SHA-1)随着计算能力提升逐渐暴露安全问题；另一方面，早期 SSL 对握手过程保护不足，容易遭受：

中间人攻击(MITM)与协议降级攻击(Downgrade Attack)。

攻击者可能诱导通信双方降级到弱密码套件，甚至利用协议漏洞(如 POODLE 攻击)解密部分通信内容。

针对这些问题，现代 TLS(尤其 TLS 1.2 / TLS 1.3)做出了显著改进：

对完整握手过程进行签名与绑定，防止中间人和降级攻击；
废弃弱加密算法(如 RC4、MD5、SHA-1)；
使用更强密码套件(如 AES-GCM、ChaCha20-Poly1305)；
引入 临时 Diffie-Hellman 密钥交换(ECDHE)，提供 前向安全性(Forward Secrecy)，即使长期密钥泄露，历史通信内容仍无法被解密。

因此，现代 TLS 在机密性、身份认证和抗攻击能力方面较 SSL有了显著提升。

临时DH密钥交换提供前向完美保密。提高了协议的通信安全。

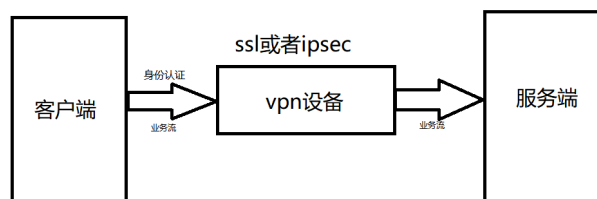
b. 在过去的二十年里，企业的标准安全架构是“边界防御”：利用 **IPSec VPN** 将外网隔离，员工接入 VPN 后，在内网利用 Kerberos 实现单点登录 (SSO) 并自由访问各种服务。[15points]

- 1). 在如今“云原生 + 居家办公”的时代，为什么传统的“VPN + 内网 Kerberos”模型变得既不安全又低效？请列举至少两个该模型在现代场景下的致命缺陷。[5 points]
- 2). 请用一两句话概括“零信任 (Zero Trust)”架构的最核心原则。在零信任架构中，传统的 IP地址还被信任吗？访问控制的决策主要基于什么？[5 points]
- 3). 随着业务迁移到公有云(如 AWS, Azure)，应用变成了无数的微服务和 Web API。在这种现代 Web 和云环境中，由于跨域和互联网环境的限制，Kerberos

已经被边缘化。请写出目前工业界取代 Kerberos, 用于实现现代跨域单点登录 (SSO) 以及 API 授权授权的最主流的三个现代协议/标准框架名称, 并简单描述它们。[5 points]

Answer:

1) 传统企业安全架构采用“边界防御(Perimeter Security)”思想, 即通过 IPSec VPN 将企业内外网隔离, 员工接入 VPN 后进入可信内网, 再通过 Kerberos 实现单点登录(SSO) 并访问内部服务。



传统VPN+内网架构

然而在“云原生 + 居家办公”的时代, 该模型暴露出多个致命缺陷:

a. 默认信任内网, 安全边界失效

传统模型默认认为: 进入 VPN 即可信。

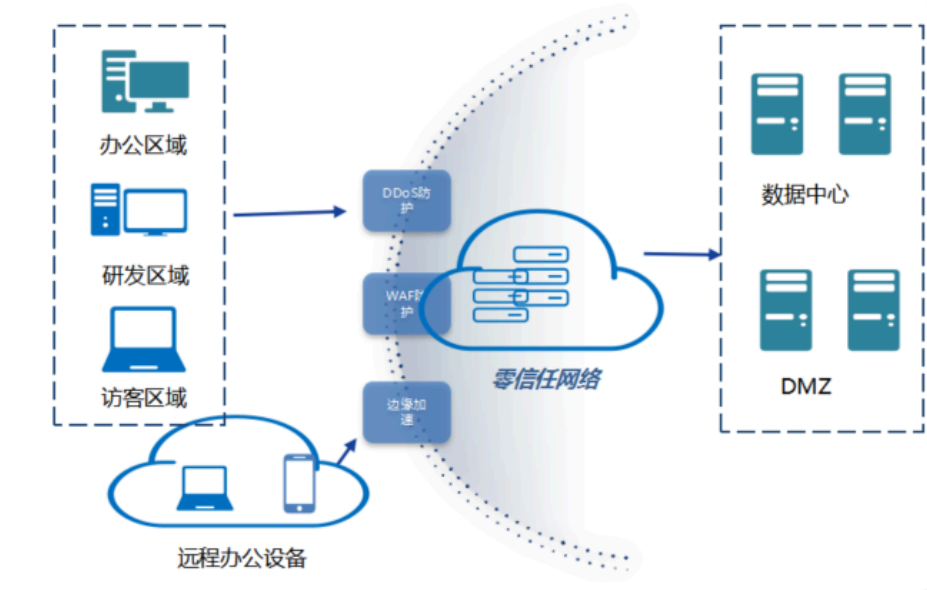
员工连接 VPN 后, 通常可自由访问大量内网资源, 一旦攻击者窃取 VPN 凭据、终端被恶意软件感染, 攻击者即可在内网进行:

横向移动;

权限提升;

大规模数据窃取。

因此, “内网可信”的假设已经不再成立。



零信任架构

2) 零信任架构的核心原则是永远不默认信任，持续验证，即：不再默认信任内网、设备或IP地址，任何访问请求都需要持续验证身份和安全状态。

在零信任架构中，传统的IP地址不再被默认信任。

访问控制决策主要基于 用户身份，设备状态，多因素认证，用户行为风险，上下文信息(时间、地理位置、风险等级)，最小权限策略。

即“基于身份和上下文，而非网络位置”进行访问控制。

3)随着应用迁移到云环境、微服务与 Web API 普及，Kerberos 因跨域和互联网环境限制逐渐边缘化。现代工业界主要采用以下三类协议/标准框架：

OAuth 2.0，一种授权框架，主要用于第三方应用授权访问API。其核心机制是通过访问令牌(Access Token)实现安全授权。

OpenID Connect (OIDC), 一种基于OAuth 2.0 的身份认证协议，用于现代跨域单点登录(SSO)。支持 Google Login, 通过用户ID 口令实现用户身份认证。

SAML, 一种基于XML的企业级单点登录标准，广泛用于企业身份联邦。其核心思想是身份提供方服务提供方传递认证断言。

3. 软件安全 [20points]

a. 什么是‘金丝雀’技术？其如何用于防止缓冲区溢出攻击？微软是如何使用该技术的(编译器的 /GS 选项)？ [10points]

Answer:

“金丝雀(Canary)”技术是一种用于检测栈缓冲区溢出(Stack Buffer Overflow)的防御机制。其核心思想是在栈中的局部缓冲区与关键控制数据(如返回地址)之间插入一个特殊的保护值,称为“金丝雀值(Canary Value)”。

当程序发生缓冲区溢出时,攻击者通常需要覆盖返回地址以劫持程序控制流。但在覆盖返回地址之前,必须先覆盖位于其前面的金丝雀值。程序会在函数返回前检查当前金丝雀值是否与原值一致:当前 Canary == 原始 Canary ?

若发现被修改,则说明发生了缓冲区溢出,程序会立即终止,从而阻止攻击者进一步覆盖返回地址并执行恶意代码。

在一些实现中,金丝雀值会包含特殊字节(如 0x00),因为许多 C 语言字符串函数(如 strcpy)会将 0x00 视为字符串结束符。这使攻击者难以通过普通字符串输入完整覆盖 Canary 而不触发检测机制,从而进一步提高缓冲区溢出攻击的难度。

现代系统(如微软 Visual C++ 的 /GS 机制)通常采用**随机生成的安全 Cookie(Security Cookie)**作为金丝雀值,而不是固定常量,以提高抗绕过能力。

b. 假如 Alice 的计算机设置了保护缓冲区溢出攻击的 NX 位。如果 Alice 的计算机中运行某些具有缓冲区溢出漏洞的程序,请问 Trudy 是否可以利用这些缓冲区溢出漏洞对 Alice 的计算机发动拒绝服务攻击?请说明原因。 [10points]

Answer:

可以。NX(No-eXecute)位的作用是防止攻击者通过缓冲区溢出在栈或堆中执行恶意代码,从而阻止远程代码执行攻击。然而,NX只能防止“执行恶意代码”,并不能防止缓冲区溢出本身发生。

攻击者仍可通过构造恶意输入覆盖程序关键控制数据(如返回地址),导致程序异常崩溃或终止服务。因此,即使启用了NX位,Trudy 仍然可以利用缓冲区溢出漏洞覆盖关键控制数据导致程序非法跳转,内存访问异常,段失效和程序崩溃等结果,导致程序异常终止,从而导致服务无法正常使用,从而对 Alice 的计算机发动拒绝服务(DoS)攻击,只是更难进一步获取系统控制权。

4. 代码实践 [30points]

a. 下面是一段使用 AES-CBC 模式的 Python 伪代码/残缺代码。[15point]

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os

key = os.urandom(32) # 256-bit key
iv = os.urandom(16) # 128-bit IV

def vulnerable_encrypt(plaintext_bytes):
    # (假设此处已包含对齐 padding 逻辑)
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    encryptor = cipher.encryptor()
    return encryptor.update(plaintext_bytes) + encryptor.finalize()

def vulnerable_decrypt(ciphertext_bytes):
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    decryptor = cipher.decryptor()
    # 严重缺陷: 解密后没有任何完整性校验就直接返回!
    return decryptor.update(ciphertext_bytes) + decryptor.finalize()
```

- 1). 请简述为什么上述 `vulnerable_decrypt` 函数是危险的? 如果攻击者在网络上拦截了密文, 随机修改了其中的几个字节再发送给接收方, 接收方系统会发生什么? [5 points]
- 2). 请查阅 Python `cryptography` 库的官方文档, 重写加密和解密函数。[10points]

要求:

- 废弃 AES-CBC, 改用现代协议推荐的 AES-GCM (Galois/Counter Mode) 模式。
- 你的解密函数必须能够捕获并处理被篡改的密文(例如抛出 `InvalidTag` 异常), 而不是返回错误的明文。
- 请提供完整的、可运行的 Python 源码。

Answer:

1) vulnerable_decrypt 函数的根本危险在于它缺乏明文的完整性校验与真实性认证。若攻击者拦截并随机篡改密文, 接收方系统将面临以下严重后果:

密文篡改导致明文损坏(错误扩散效应): 由于 AES-CBC 模式的链式特性,

$$C_i = E_K(P_i \oplus C_{i-1})$$

若密文第 n 块被篡改:解密后对应的当前明文块(第 n 块) 将彻底退化为随机的垃圾乱码。紧随其后的下一个明文块(第 $n+1$ 块) 在相同字节位置上也会遭受致命的异或损坏。

由于缺乏校验, 函数会盲目地将包含乱码的脏数据直接返回给上层业务, 导致业务逻辑出错。

引发可用性危机(DoS 攻击): 若篡改涉及密文的最后一个数据块, 通常会直接破坏 PKCS#7 填充(Padding) 结构。解密库在检测到不合法的填充格式时会抛出填充异常(如 ValueError)。如果程序内部未预先捕获并处理该异常, 将导致整个后端系统进程崩溃, 引发拒绝服务(DoS)。

面临 Padding Oracle 侧信道攻击:即使程序设置了异常处理, 如果编写不当(例如根据“填充错误”和“解密成功但明文不符”返回不同的错误代号或 HTTP 状态码), 攻击者就可以利用这种报错差异构建“侧信道攻击”。通过盲测并观察系统的报错反应, 攻击者可以在极短时间内逆向破解出完整的明文, 导致加密形同虚设。

发送篡改密文, 服务器返回 **HTTP 500**

用户验证不通过 服务器返回 **HTTP 401**

2) 在AES-GCM中, 解密函数内部是先进行校验, 然后才输出, 如果进行完整性校验时, 无法通过, 则抛出InvalidTag错误并切断后续流程, 不会像CBC那样直接返回带有乱码的原文给上层业务。

AES-GCM中的GCM(Galois/Counter Mode) 是使用流密码思想与校验码的结合, 所以不需要进行填充(padding)。所以优化了之前出现的填充错误和对应的攻击。

AES-GCM解密函数中使用了对应的异常处理程序(try: except:)。通过捕捉异常, 将未知的攻击通过异常处理变为业务逻辑返回, 保证服务的高可靠性, 鲁棒性。

具体代码在代码文件中

b. 一位初级后端开发工程师为公司的内部系统编写了一个简单的登录验证模块。该模块使用 SQLite 数据库存储用户，并使用了 MD5 对用户密码进行“加密”存储。[15point]

```
import sqlite3
import hashlib

def init_db():
    conn = sqlite3.connect(':memory:') # 使用内存数据库方便测试
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, username TEXT, password_hash TEXT)")
    # 预置一个管理员账号，密码为 admin123 的 MD5 值
    cursor.execute("INSERT INTO users (username, password_hash) VALUES ('admin', '0192023a7bbd73250516f069df18b500')")
    conn.commit()
    return conn

def insecure_login(conn, username, password):
    # 开发者:我把密码 MD5 了一次，数据库里看不到明文，很安全！
    pwd_hash = hashlib.md5(password.encode()).hexdigest()

    # 开发者:直接拼接到 SQL 语句里查询
    query = f"SELECT * FROM users WHERE username='{username}' AND password_hash='{pwd_hash}'"

    cursor = conn.cursor()
    cursor.execute(query)
    user = cursor.fetchone()

    if user:
        print(f"登录成功！欢迎回来，{user[1]}。")
        return True
    else:
        print("登录失败：用户名或密码错误。")
        return False
```

- 1). 在不知道管理员密码的情况下，请构造一个恶意的 `username` 字符串(假设前端未做任何过滤)。当该字符串传入 `insecure_login` 时，能够触发 SQL 注入，直接绕过密码验证，打印出“登录成功”。请写出这个 payload。[5 points]
- 2). 修改 `insecure_login` 函数中的数据库查询逻辑。请使用参数化查询 (Parameterized Queries) 技术重写代码，彻底杜绝 SQL 注入漏洞。[10 points]

要求：

- 请提供完整的、可运行的 Python 源码

Answer:

- 1) 使用payload 1:username: "admin' -", password: "****", 将此用户名和任意密码输入之后，带入数据库语句为：

```
SELECT * FROM users WHERE username='admin' -- ' AND password_hash='{pwd_hash}'
```

其中 用户名 admin' -- 中的单引号和代码中的单引号完成用户名的输入, 后面的 -- 为SQL语句中用于单行注释的符号。

执行后, 语句只检验前面的admin用户名是否存在, 而不检验后面的密码校验。

从而实现SQL注入, 绕过密码认证。

使用payload 2: username: "' OR 1=1 --", password: "****", 将此用户名和任意密码输入之后, 带入数据库语句为:

```
SELECT * FROM users WHERE username='' OR 1=1 -- ' AND password_hash='{pwd_hash}'
```

当前代码语句, 在不确定数据库中是否有admin用户的情况下, 依然可以成功登录。通过添加OR 后面的恒真语句可以实现当前语句恒为真, 从而登录成功。

2) 为了杜绝SQL注入, 不使用字符串拼接形式, 而是使用参数化查询, 让数据库驱动把输入仅仅当作数据处理, 而不是“可执行的代码”。

具体代码在代码文件中。